

דף נוסחאות והגדרות לבחינה - הנדסת נתונים

מבוסס על נושאי הקורס: מערכות מבוזרות, ETL/DWH, MapReduce, טקסט, סטרימינג, מאפיינים, DataOps ו-SQL.

מתקדמות בחפיפה, גם אם לא בו-זמנית.

תהליך מול חוט: תהליך מבודד יותר וכבד יותר; חוט קל יותר אך חולק זיכרון ולכן חשוף למצבי מרוץ.

מודל עבודה ומסלול קריטי:

$$T_p \geq \max\left(\frac{W}{p}, S\right)$$

$$\text{Speedup} \leq \frac{W}{S}$$

W - כלל העבודה; S - המסלול הקריטי; p - מספר עובדים.

חוק אמדל:

$$\text{Speedup}(p) = \frac{1}{(1 - \alpha) + \frac{\alpha}{p}}$$

α - החלק שניתן למקבל. כאשר $p \rightarrow \infty$, p - ההאצה חסומה על ידי החלק הסדרתי.

Skew: התפלגות לא מאוזנת של מפתחות או מחיצות. מדד שימושי:

$$\text{SkewRatio} = \frac{\max(\text{size partition})}{\text{median}(\text{size partition})}$$

טיפול ב-skew: אגרגציה מקומית, salting, מחלק מותאם אישית, שינוי מפתח חלוקה, או פיצול מפתח חם לשני שלבים.

4. ETL, קליטה ואידמפוטנטיות

ETL: חילוף, טרנספורמציה ואז טעינה. טוב כאשר רוצים בקרת איכות לפני המחסן.

ELT: חילוף וטעינה קודם, טרנספורמציה בתוך המחסן. טוב כאשר למחסן יש כוח חישוב גדול ורוצים לשמור נתונים גולמיים.

CDC: לכידת שינויים בלבד: DELETE, UPDATE, INSERT. מפחית טעינות מלאות.

Watermark: נקודת זמן או מזהה שמגדירים עד איפה נקראו נתונים בהצלחה. כלל חשוב: מקדמים watermark רק אחרי פרסום מוצלח.

גבול קריאה בטוח:

$$\text{upper_bound} = \text{now} - \text{safety_buffer}$$

ה-safety buffer מונע דילוג על רשומות שהגיעו מאוחר למקור.

אידמפוטנטיות:

$$f(f(D)) = f(D)$$

הרצה חוזרת על אותו קלט לא משנה את מצב היעד. מממשים עם UPSERT, MERGE, או overwrite למחיצה מוגדרת.

דה-דופליקציה דטרמיניסטית: בוחרים מפתח עסקי, כלל שבירת שוויון יציב, ושורד יחיד לכל מפתח.

טבלת בקרה: upper_bound, watermark, job_key, run_id, status, rows_loaded, rows_read, error_code. מצבים טיפוסיים: STARTED -> VALIDATED -> LOADED -> PUBLISHED; כשל עובר ל-FAILED.

5. מחסן נתונים, אגם נתונים ומידול

DWH: שכבה מובנית ואוצרת לדוחות ו-BI. לרוב schema-on-write.

Data Lake: אחסון גולמי או חצי-מעובד בקנה מידה גדול. לרוב schema-on-read.

טבלת Fact: מדדים כמותיים ברמת גרעין מוגדרת. **טבלת Dimension:** מאפיינים לתיאור, סינון וחיתוך.

גרעין Fact: הרמה המדויקת של שורה אחת בטבלת העובדות. בלי גרעין ברור נוצרים מדדים כפולים.

Star schema: טבלת עובדות מרכזית ומסביבה מימדים. טוב לשאליות BI. **Snowflake schema:** מימדים מנורמלים יותר; פחות כפילות, יותר joins.

1. עקרונות יסוד וצינורות נתונים

הנדסת נתונים: תכנון, בנייה ותפעול של צינורות נתונים אמנים, מדרגיים וניתנים לניטור, כך שנתונים גולמיים הופכים לנתונים שימושיים לצריכה אנליטית או מודלית.

צינור נתונים בסיסי: מקור -> קליטה -> אחסון גולמי -> טרנספורמציה -> שכבה אוצרת -> צריכה.

מודל זמן כולל:

$$T_{total} = T_{ingest} + T_{transform} + T_{aggregate} + T_{load}$$

או, בגרסת ETL:

$$T_{total} = T_{extract} + T_{transform} + T_{load}$$

קיבולת אחסון:

$$S = \frac{V_d \cdot R \cdot r}{c}$$

V_d - נפח יומי; R - מספר ימי שמירה; r - פקטור שכפול; c - on דחיסה.

עלות העברה ברשת: בקירוב, NetworkBytes = מספר רשומות $\times$ גודל רשומה. העברה בין אזורים או בין צמתים עלולה להיות צוואר בקבוק.

Batch מול Streaming: Batch פשוט וזול יותר אך בעל שיהוי גבוה; Streaming מתאים ל-SLA נמוך אך דורש חלונות זמן, מצב, טיפול באיחורים וכתביה אידמפוטנטית.

2. מסדי נתונים מבוזרים

חלוקה למחיצות: פיזור נתונים בין צמתים לפי מפתח.

$$\text{partition}(k) = \text{hash}(k) \bmod R$$

עומס נתונים לצומת:

$$S_{node} = \frac{D \cdot r}{N}$$

D - נפח נתונים; r - מספר עותקים; N - מספר צמתים.

זמינות בקירוב עם שכפול:

$$P_{avail} \approx 1 - p^r$$

p - הסתברות כשל של צומת; r - פקטור שכפול.

Replication: שמירת עותקים של אותם נתונים לצורך זמינות וקריאה מהירה. המחיר: עלות אחסון, כתיבה, סנכרון וסיכון לקריאות ישנות.

Stale read: קריאה מרפליקה שעדיין לא קיבלה כתיבה חדשה. המדד המרכזי: replication lag.

SQL מול NoSQL:

• SQL: טוב ליחסים, JOIN, עקביות ועסקאות; יקר יותר ב-cross-partition joins.

• NoSQL: טוב לגישה לפי מפתח, זמינות וסקייל אופקי; דורש דנורמליזציה ולוגיקה אפליקטיבית.

ACID: אטומיות, עקביות, בידוד, עמידות. מתאים לטרנזקציות קריטיות.

BASE: זמינות בסיסית, מצב רך, עקביות בסוף התהליך. מתאים לסקייל גבוה עם פשרות עקביות.

CAP: בזמן חלוקת רשת לא ניתן לקבל גם עקביות חזקה וגם זמינות מלאה. בוחרים CP או AP.

3. מקביליות, חלוקה ועומס

מקביליות: משימות רצות בו-זמנית בפועל. **קונקרנציה:** משימות

SCD Type 2: שמירת היסטוריה של מימד עם valid_from, valid_to, is_current ומפתח סרוגייט.

עלות סריקה עם **partition pruning**:

$$\text{ScanCost} = s \cdot |F|$$

$|F|$ - גודל טבלת העובדות; s - חלק המחיצות שנקרא. אם $s = 0.05$, נסרקים רק 5% מהנתונים.

עלות **join אינטואיטיבית**:

$$\text{JoinWork} \approx O(|F_{\text{pruned}}| + \sum_i |D_i|)$$

ה-Fact המסונן בדרך כלל דומיננטי. מימדים גדולים יכולים לגרום **shuffle ו-spill**.

כלל בחינה: תמיד לציין גרעין, מפתחות, מחיצות, מימדים, מדדים, וסיכון של סריקה מלאה.

6. MapReduce

מודל: Map פולט זוגות (k_2, v_2) ; Shuffle מקבץ לפי מפתח; Reduce מחשב תוצאה לכל מפתח.

עלות **Shuffle**:

$$C_{\text{shuffle}} = E \cdot s$$

E - מספר זוגות שנפלטו מה-map; s - גודל זוג ממוצע. מפתחים עלות בעזרת combiner, סינון מוקדם, קידוד קומפקטי ואגרגציה מקומית.

פירוק זמן ריצה:

$$T_{\text{total}} = T_{\text{map}} + T_{\text{shuffle}} + T_{\text{reduce}}$$

בפועל, shuffle וה-reducer הגדול ביותר קובעים את זמן הסיום. **Combiner**: אגרגציה מקומית לפני shuffle. בטוח רק לפעולות אסוציאטיביות וקומוטטיביות כגון סכום, ספירה, מינימום ומקסימום. ממוצע דורש לשמור $(sum, count)$ ולא ממוצע מקומי בלבד.

Partitioner: קובע לאיזה Reducer נשלח כל מפתח. ברירת מחדל: $\text{hash}(\text{key}) \bmod R$.

Reduce-side join: שני מקורות פולטים אותו מפתח הצטרפות. המחיר: shuffle של שני הצדדים. אם צד קטן נכנס לזיכרון, שקלו **broadcast/map-side join**.

תבניות נפוצות: word count; אינדקס הפוך; סינון במפה; מכפלת מטריצה-וקטור בשני שלבים; PageRank.

7. אלגוריתמים מבוזרים

מכפלת מטריצה-וקטור:

$$y_i = \sum_j A_{i,j} \cdot v_j$$

שלב 1: מפתח j , מצמידים ערכי $A_{i,j}$ לערך v_j ופולטים $(i, A_{i,j} \cdot v_j)$.
שלב 2: מקבצים לפי i וסוכמים.

PageRank עם דעיכה:

$$\text{PR}_1(p) = \frac{1-d}{N} + d \left(\sum_{q \rightarrow p} \frac{\text{PR}_0(q)}{|\Gamma(q)|} + \frac{M}{N} \right)$$

d - מקדם דעיכה; N - מספר דפים; $\Gamma(q)$ - קישורים יוצאים מ- q ; M - מסה של צמתים תלויים ללא קישורים יוצאים.

כלל: ב-PageRank מותר לשלב תרומות מספריות ב-combiner, אך אסור לאבד את רשימת השכנים.

8. טקסט, TF-IDF, n-grams ודמיון

טוקניזציה: פירוק טקסט למונחים. חובה להגדיר ניקוי, אותיות גדולות/קטנות, סימני פיסוק, עצירות, ושפת מקור.

TF:

$$\text{tf}(t, d) = \frac{f_{t,d}}{|d|}$$

$f_{t,d}$ - מספר הופעות המונח t במסמך d ; $|d|$ - אורך המסמך.

DF:

$$\text{df}(t) = |\{d : t \in d\}|$$

IDF:

$$\text{idf}(t) = \log \left(\frac{N+1}{\text{df}(t)+1} \right)$$

בתרגילים ייתכן גם $\log(N/\text{df})$. חשוב לציין בסיס לוג אם נדרש.

TF-IDF:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \text{idf}(t)$$

גבוה כאשר מונח נפוץ במסמך אך נדיר בקורפוס.

מספר n-grams במסמך באורך L:

$$\max(L - n + 1, 0)$$

גודל מילון אפשרי:

$$|V_n| \leq |V|^n$$

מודל שפה n-gram:

$$P(w_{1:T}) \approx \prod_{t=1}^T P(w_t | w_{t-n+1:t-1})$$

$$\hat{P}(w | h) = \frac{c(h, w)}{c(h)}$$

דמיון קוסינוס:

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$$

ערך קרוב ל-1 מצביע על כיוון דומה של וקטורים.

9. סטרימינג ואלגוריתמי קירוב

מודל זרם: כל איבר נקרא פעם אחת או מעט פעמים, והזיכרון מוגבל.

לכן לעיתים משתמשים ב-sketch.

תוחלת ושונות:

$$E[X] = \sum_i x_i P(X = x_i)$$

$$V[X] = E[X^2] - (E[X])^2$$

קירוב הסתברותי:

$$P(|\hat{X} - X| \leq \epsilon X) \geq 1 - \delta$$

ϵ - שגיאה יחסית; δ - הסתברות כשל.

Union bound:

$$P(E_1 \cup \dots \cup E_n) \leq \sum_i P(E_i)$$

Markov:

$$P(X > (1 + \epsilon)\mu) \leq \frac{1}{1 + \epsilon}$$

Chebyshev:

$$P(|X - \mu| \geq \epsilon\mu) \leq \frac{\sigma^2}{\mu^2 \epsilon^2}$$

Morris counter: מתחילים $X = 0$; עבור כל איבר מגדילים את X בהסתברות 2^{-X} .

$$\hat{n} = 2^X - 1$$

גרסת ממוצע עם s מונים:

$$\hat{n} = \frac{1}{s} \sum_i (2^{X_i} - 1), \quad s \geq \frac{1}{\delta \epsilon^2}$$

Distinct count F_0 : מספר ערכים שונים בזרם. חישוב מדויק דורש זיכרון גדול; FM/HLL נותנים קירוב.

HyperLogLog

$$\text{Error Relative} \approx \frac{1.04}{\sqrt{m}}$$

m - מספר רגיסטרים.

Count-Min Sketch: מעריך תדירות לכל מפתח, נוטה להערכת יתר בגלל התנגשויות. מתאים לזיהוי heavy hitters.

חלונות זמן: tumbling לא חופפים; sliding חופפים; session תלויים בפעילות משתמש.

Event time לעומת processing time: זמן האירוע עצמו מול זמן הגעתו למערכת. Watermark מגדיר עד מתי מצפים לאירועים מאוחרים.

10. הנדסת מאפיינים מאפיין נכון בזמן:

$$f(e, t) = g(\{x \mid x.entity = e, x.ts \leq t\})$$

אסור להשתמש במידע אחרי זמן החיזוי.

עקביות אימון-שירות:

$$f_{train}(e, t) = f_{serve}(e, t)$$

אותו חישוב ואותן הגדרות חייבים לשמש באימון ובפרודקשן.

אחסון מאפיינים:

$$\text{feature_storage} \approx N_{entities} \times N_{asof} \times \text{bytes_per_row}$$

נרמול Min-Max:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

סטנדרטיזציה:

$$z = \frac{x - \mu}{\sigma}$$

One-hot: עמודה בינארית לכל קטגוריה. מתאים לקרדינליות נמוכה. **Hashing trick**: ממפה קטגוריות למספר קבוע של ממדים, עם סיכון להתנגשויות.

Target encoding: החלפת קטגוריה בממוצע יעד. חובה להשתמש בחלוקה תקינה או out-of-fold כדי למנוע דליפת יעד.

עבודת backfill מול אינקרמנטלי:

$$Work_{backfill} = O(|D| \cdot T), \quad Work_{incremental} = O(|\Delta|)$$

11. DataOps, איכות וניטור

DataOps: אוטומציה, בדיקות, ניטור ושחרור מבוקר של צינורות נתונים.

ממדי איכות: שלמות, תקפות, ייחודיות, עקביות, עדכניות, דיוק.

רענונות / שיהוי נתונים:

$$L_t = \text{now} - \max(\text{event_ts}_t)$$

חריגה לפי ציון תקן:

$$z_t = \frac{|N_t - \mu|}{\sigma}$$

N_t - מספר רשומות או מדד בזמן t ; μ, σ - בסיס היסטורי.

שיעור מעבר בדיקות:

$$\text{PassRate} = \frac{\#passed}{\#total}$$

שיעור כשל:

$$\text{FailureRate} = \frac{\#failed}{\#total}$$

Quality gate: אין פרסום לשכבה צרכנית אם בדיקות קריטיות נכשלות. בדיקות לא קריטיות יכולות להתריע בלי לחסום.

מדדי תפעול: זמן ריצה, p50/p95/p99, רענונות, כמות שורות, אחוז DLQ, כפילויות, SLA שהוחמצו, עלות סריקה, retry count.

12. SQL מתקדם ואופטימיזציה

DML: CREATE, DELETE, UPDATE, INSERT, SELECT. **DDL**: ALTER, DROP.

סוגי JOIN

- INNER JOIN: רק התאמות בשני הצדדים.
- LEFT JOIN: כל צד שמאל, ו-NULL כשאין התאמה בימין.
- RIGHT JOIN: כל צד ימין.
- FULL OUTER JOIN: כל השורות משני הצדדים.

NULL: אינו מחרוזת ואינו שווה לכלום, גם לא לעצמו. משתמשים ב-IS NULL ולא ב=NULL.

חלונות ב-SQL:

func(...) OVER (PARTITION BY ... ORDER BY ...)

PARTITION BY מחלק קבוצות; ORDER BY קובע סדר בתוך חלון. **פונקציות דירוג**: ROW_NUMBER נותן מספר ייחודי; RANK משאיר חורים אחרי תיקו; DENSE_RANK לא משאיר חורים; NTILE(n) מחלק לדליים.

פונקציות ערך: LAG ערך קודם; LEAD ערך הבא; LAST_VALUE/FIRST_VALUE לפי חלון וסדר.

כללי אופטימיזציה: סנונו מוקדם ב-WHERE; השתמשו במפתחות join; הימנעו מסריקה מלאה; דחפו פילטר מחיצה; בדקו קרדינליות; אל תייצרו join explosion.

13. תבנית תשובה לבחינה

1. הגדירו דרישה: SLA, עקביות, נפח, קצב, עלות, רענונות.
2. בחרו ארכיטקטורה: batch/stream, SQL/NoSQL, ETL/ELT, DWH/lake.
3. כתבו נוסחה או אומדן: זמן, אחסון, shuffle, סריקה, זיכרון או שגיאה.
4. ציינו כשל סביר: כפילויות, skew, איחור נתונים, דליפת מאפיינים, סריקה מלאה, רגרסיה שקטה.
5. הוסיפו מנגנון הגנה: watermark, MERGE, quality gate, ניטור, salting, combiner, DLQ p95/p99.